

Indexing Data Frames

Feb 05, 2025

Short Refresher on Indexing Vectors

```
data(islands)
length(islands)
```

```
## [1] 48
```

```
islands
```

##	Africa	Antarctica	Asia	Australia
##	11506	5500	16988	2968
##	Axel Heiberg	Baffin	Banks	Borneo
##	16	184	23	280
##	Britain	Celebes	Celon	Cuba
##	84	73	25	43
##	Devon	Ellesmere	Europe	Greenland
##	21	82	3745	840
##	Hainan	Hispaniola	Hokkaido	Honshu
##	13	30	30	89
##	Iceland	Ireland	Java	Kyushu
##	40	33	49	14
##	Luzon	Madagascar	Melville	Mindanao
##	42	227	16	36
##	Moluccas	New Britain	New Guinea	New Zealand (N)
##	29	15	306	44
##	New Zealand (S)	Newfoundland	North America	Novaya Zemlya
##	58	43	9390	32
##	Prince of Wales	Sakhalin	South America	Southampton
##	13	29	6795	16
##	Spitsbergen	Sumatra	Taiwan	Tasmania
##	15	183	14	26

Index a vector with its INDEX (always an INTEGER)

```
islands[3]
```

```
## Asia
```

```
## 16988
```

```
islands[3:6]
```

```
##      Asia      Australia Axel Heiberg      Baffin
##      16988           2968           16       184
```

```
islands[-c(1,2)] #REMOVE values "Africa" and "Antarctica"
```

```
##      Asia      Australia      Axel Heiberg      Baffin
##      16988           2968           16       184
##      Banks      Borneo      Britain      Celebes
##      23          280          84          73
##      Celon      Cuba      Devon      Ellesmere
##      25          43          21          82
##      Europe      Greenland      Hainan      Hispaniola
##      3745          840          13          30
```

3/39

Index a vector by NAME

```
islands["Asia"]
```

```
## Asia
```

```
## 16988
```

```
islands[c("Asia", "Java")]
```

```
## Asia Java
```

```
## 16988 49
```

4/39

Index a vector with LOGICALS

First create a logical vector

```
log.vec<-islands > 100 & islands < 1000 # TRUE for islands > 100 & less than 1000
str(log.vec)
```

```
## Named logi [1:48] FALSE FALSE FALSE FALSE FALSE TRUE ...
## - attr(*, "names")= chr [1:48] "Africa" "Antarctica" "Asia" "Australia" ...
```

log.vec

##	Africa	Antarctica	Asia	Australia
##	FALSE	FALSE	FALSE	FALSE
##	Axel Heiberg	Baffin	Banks	Borneo
##	FALSE	TRUE	FALSE	TRUE
##	Britain	Celebes	Celon	Cuba
##	FALSE	FALSE	FALSE	FALSE
##	Devon	Ellesmere	Europe	Greenland
##	FALSE	FALSE	FALSE	TRUE
##	Hainan	Hispaniola	Hokkaido	Honshu
##	FALSE	FALSE	FALSE	FALSE
##	Iceland	Ireland	Java	Kyushu

5/39

Index a vector with LOGICALS

```
islands[log.vec]
```

##	Baffin	Borneo	Greenland	Madagascar	New Guinea	Sumatra
##	184	280	840	227	306	183

6/39

Also remember “Recycling”

```
str(islands) # islands is 48 length
```

```
## Named num [1:48] 11506 5500 16988 2968 16 ...
```

```
## - attr(*, "names")= chr [1:48] "Africa" "Antarctica" "Asia" "Australia" ...
```

```
islands[c(T,F,F,F)] # Our logical vector is 4. Returns every FOURTH element.
```

```
##           Africa      Axel Heiberg           Britain           Devon
##           11506              16              84              21
##           Hainan           Iceland           Luzon           Moluccas
##              13              40              42              29
## New Zealand (S) Prince of Wales   Spitsbergen Tierra del Fuego
##              58              13              15              19
```

R knows to **recycle**.

7/39

Indexing Matrices and Data frames

Indexing matrices and data frames

Indexing vectors is a lot like indexing all other R objects.

- You still use `[`
- You can access values by name, index, or logicals

One difference:

- Use the format `x[a,b]`
 - where `x` is a matrix or data frame object,
 - `a` is the *row* index,
 - `b` is the *column* index.

9/39

Indexing Matrices by INDEX

Let's work with the `volcano` data set.

```
data(volcano)
dim(volcano)
```

```
## [1] 87 61
```

```
class(volcano)
```

```
## [1] "matrix" "array"
```

10/39

Indexing Matrices by INDEX

Use the `matrix[row, column]` format:

```
volcano[1:6, 1:10] # show the first 6 rows and first 10 columns
```

```
##      [,1] [,2] [,3] [,4] [,5] [,6] [,7] [,8] [,9] [,10]
## [1,]  100  100  101  101  101  101  101  100  100  100
## [2,]  101  101  102  102  102  102  102  101  101  101
## [3,]  102  102  103  103  103  103  103  102  102  102
## [4,]  103  103  104  104  104  104  104  103  103  103
## [5,]  104  104  105  105  105  105  105  104  104  103
## [6,]  105  105  105  106  106  106  106  105  105  104
```

11/39

Indexing Matrices by INDEX

Since we know we have 87 rows and 61 columns we know what possible values we can use to index `volcano`.

```
volcano[52:58, 20:25]
```

```
##      [,1] [,2] [,3] [,4] [,5] [,6]
## [1,]  156  159  159  160  162  162
## [2,]  155  156  157  158  159  159
## [3,]  153  154  156  157  157  157
## [4,]  152  154  155  155  155  155
## [5,]  152  153  153  153  153  153
## [6,]  151  151  151  151  151  151
## [7,]  150  150  150  150  150  150
```

Notice what R does when we don't have row names.

12/39

Indexing Matrices by INDEX

```
# Returns specified rows within ALL COLUMNS  
volcano[52:58,] # Leave out index for columns
```

```
##      [,1] [,2] [,3] [,4] [,5] [,6] [,7] [,8] [,9] [,10] [,11] [,12] [,13] [,14]  
## [1,] 115 116 117 118 119 120 121 122 123 125 128 131 134 137  
## [2,] 114 115 116 116 118 119 120 121 122 126 129 132 135 137  
## [3,] 113 114 115 116 117 118 119 120 123 126 129 132 135 138  
## [4,] 112 113 114 115 116 117 119 120 122 126 130 133 136 138  
## [5,] 111 112 114 115 116 117 118 120 122 125 131 134 137 139  
## [6,] 111 112 113 115 115 116 117 119 121 126 131 135 138 140  
## [7,] 112 113 113 114 115 116 117 119 122 127 132 135 139 141  
##      [,15] [,16] [,17] [,18] [,19] [,20] [,21] [,22] [,23] [,24] [,25] [,26]  
## [1,] 139 142 145 149 152 156 159 159 160 162 162 161  
## [2,] 140 143 146 149 152 155 156 157 158 159 159 159  
## [3,] 140 143 146 148 151 153 154 156 157 157 157 157  
## [4,] 141 143 146 148 150 152 154 155 155 155 155 155  
## [5,] 142 144 146 148 150 152 153 153 153 153 153 153  
## [6,] 142 144 146 148 150 151 151 151 151 151 151 151  
## [7,] 143 145 147 149 150 150 150 150 150 150 150 150  
##      [,27] [,28] [,29] [,30] [,31] [,32] [,33] [,34] [,35] [,36] [,37] [,38]  
## [1,] 161 160 159 158 157 155 153 150 148 146 145 143  
## [2,] 158 158 157 155 153 151 150 149 147 146 145 144
```

13/39

Indexing Matrices by INDEX

```
# Returns specified columns within ALL ROWS  
volcano[,20:25] # Leave out index for columns
```

```
##      [,1] [,2] [,3] [,4] [,5] [,6]  
## [1,] 102 101 101 102 103 104  
## [2,] 103 102 102 103 105 106  
## [3,] 104 104 105 106 107 108  
## [4,] 106 106 107 108 110 111  
## [5,] 108 109 110 112 114 115  
## [6,] 112 113 115 116 118 119  
## [7,] 116 118 120 121 122 123  
## [8,] 120 122 124 125 127 128  
## [9,] 125 127 129 130 132 134  
## [10,] 129 133 136 138 140 141  
## [11,] 137 141 143 145 146 148  
## [12,] 146 149 149 151 153 154  
## [13,] 154 155 155 157 159 161  
## [14,] 161 161 162 163 165 166  
## [15,] 163 165 168 170 171 172  
## [16,] 165 168 170 174 176 179  
## [17,] 166 170 174 178 180 182  
## [18,] 169 172 176 181 183 184
```

14/39

Indexing Matrices by NAME

```
mat<-matrix(1:4, 2)
colnames(mat)<-c("A", "B")
rownames(mat)<-c("X", "Y")
mat
```

```
##   A B
## X 1 3
## Y 2 4
```

```
mat["Y",]
```

```
## A B
## 2 4
```

15/39

Indexing dataframes

Let's look at the `dune.env` data set.

```
## install.packages("vegan") #IF you don't have it already
library(vegan)
```

```
data(dune.env)
#head(dune.env, 4) # Can also use head with str()
str(dune.env)
```

```
## 'data.frame':   20 obs. of  5 variables:
## $ A1      : num  2.8 3.5 4.3 4.2 6.3 4.3 2.8 4.2 3.7 3.3 ...
## $ Moisture : Ord.factor w/ 4 levels "1"<"2"<"4"<"5": 1 1 2 2 1 1 1 4 3 2 ...
## $ Management: Factor w/ 4 levels "BF","HF","NM",...: 4 1 4 4 2 2 2 2 1 ...
## $ Use       : Ord.factor w/ 3 levels "Hayfield"<"Haypastu"<...: 2 2 2 2 1 2 3 3 1 1 ...
## $ Manure    : Ord.factor w/ 5 levels "0"<"1"<"2"<"3"<...: 5 3 5 5 3 3 4 4 2 2 ...
```

16/39

Indexing dataframes by row

There's a huge difference between accessing a data frame by row *index* vs row *name*.

```
dune.env[c(1,2,3),] # Access by INDEX
```

```
##      A1 Moisture Management      Use Manure
## 1 2.8          1          SF Haypastu      4
## 2 3.5          1          BF Haypastu      2
## 3 4.3          2          SF Haypastu      4
```

```
dune.env[c("1", "2", "3"),] # Access by ROWNAME
```

```
##      A1 Moisture Management      Use Manure
## 1 2.8          1          SF Haypastu      4
## 2 3.5          1          BF Haypastu      2
## 3 4.3          2          SF Haypastu      4
```

17/39

Indexing dataframes by INDEX

```
data(varespec)
```

```
varespec[1:10, 1:5] # Taking a look at the upper left chunk of data
```

```
##      Callvulg Empenigr Rhodtome Vaccmyrt Vaccviti
## 18      0.55      11.13      0.00      0.00      17.80
## 15      0.67       0.17      0.00      0.35      12.13
## 24      0.10       1.55      0.00      0.00      13.47
## 27      0.00      15.13      2.42      5.92      15.97
## 23      0.00      12.68      0.00      0.00      23.73
## 19      0.00       8.92      0.00      2.42      10.28
## 22      4.73       5.12      1.55      6.05      12.40
## 16      4.47       7.33      0.00      2.15       4.33
## 28      0.00       1.63      0.35     18.27       7.13
## 13     24.13       1.90      0.07      0.22       5.30
```

18/39

Indexing dataframes by INDEX

```
varespec[c(1,2,3), 1:8] # Access by INDEX. (Same as varespec[1:3, 1:8])
```

```
##      Callvulg Empenigr Rhodtome Vaccmyrt Vaccviti Pinusylv Descflex Betupube
## 18      0.55      11.13         0      0.00      17.80      0.07         0         0
## 15      0.67       0.17         0      0.35      12.13      0.12         0         0
## 24      0.10       1.55         0      0.00      13.47      0.25         0         0
```

19/39

Indexing dataframes by ROWNAME

```
varespec[c("1", "2", "3"), 1:8] # Access by ROWNAME
```

```
##      Callvulg Empenigr Rhodtome Vaccmyrt Vaccviti Pinusylv Descflex Betupube
## NA         NA         NA         NA         NA         NA         NA         NA
## 2      0.05      9.30         0         0      8.50      0.03         0         0
## 3      0.03      3.65         0         0      4.43      0.00         0         0
```

```
rownames(varespec) # "1" is not a valid row name in `varespec`
```

```
## [1] "18" "15" "24" "27" "23" "19" "22" "16" "28" "13" "14" "20" "25" "7"  "5"
## [16] "6"  "3"  "4"  "2"  "9"  "12" "10" "11" "21"
```

This should be a **warning** to you about what to assign row names to avoid potentially big errors. When possible, assign descriptive, unique, short row names. Often times this is a sample ID.

20/39

Remember the **Birds** data frame?

Birds

```
##           Type Num. Extinct
## Eagle   BirdofPrey    3  FALSE
## Hawk    BirdofPrey   15  FALSE
## Bluebird SongBird   42  FALSE
## Egret    Shorebird    8  FALSE
## Dodo     Flightless    0   TRUE
## Blue Jay SongBird   29  FALSE
## Kiwi     Flightless   17  FALSE
```

Each rowname is a unique, descriptive character identifier.

21/39

Indexing dataframes by column

You *can* index **columns** by column index, but ...

```
dune.env[1:3, 3:4]
```

```
##   Management      Use
## 1          SF Haypastu
## 2          BF Haypastu
## 3          SF Haypastu
```

... it's preferable to use column names when possible—more clear, more *explicit*.

```
dune.env[1:3, c("Management", "Use")]
```

```
##   Management      Use
## 1          SF Haypastu
## 2          BF Haypastu
## 3          SF Haypastu
```

22/39

Accessing whole vectors within a data frame

Use the `$`. After this the result can be worked with just like a vector by appending arguments within `[]`.

```
dune.env$A1
```

```
## [1] 2.8 3.5 4.3 4.2 6.3 4.3 2.8 4.2 3.7 3.3 3.5 5.8 6.0 9.3 11.5  
## [16] 5.7 4.0 4.6 3.7 3.5
```

```
dune.env$A1[1:5]
```

```
## [1] 2.8 3.5 4.3 4.2 6.3
```

In this form, you have to use the column name.

23/39

Accessing whole vectors within a data frame

Use the `$`. After this the result can be worked with just like a vector by appending arguments within `[]`.

```
dune.env$A1[1:5]
```

```
## [1] 2.8 3.5 4.3 4.2 6.3
```

```
dune.env[1:5, "A1"]
```

```
## [1] 2.8 3.5 4.3 4.2 6.3
```

24/39

Using the **varespec** dataset, find the difference in cover values between 'Vaccviti' and 'Cladrang' at each site.

25/39

Using the **varespec** dataset, find the difference in cover values between 'Vaccviti' and 'Cladrang' at each site.

```
varespec$Vaccviti-varespec$Cladrang
```

```
## [1] -3.67  4.00  7.95  8.34 14.51  5.33  8.80 -9.70  6.26 -18.37
## [11]  3.98  2.91  8.35 -34.30 -56.25 -27.00 -25.07 -18.05 -16.00 16.50
## [21]  1.20  3.20 -16.90 20.30
```

OR

```
varespec[, "Vaccviti"]-varespec[, "Cladrang"]
```

```
## [1] -3.67  4.00  7.95  8.34 14.51  5.33  8.80 -9.70  6.26 -18.37
## [11]  3.98  2.91  8.35 -34.30 -56.25 -27.00 -25.07 -18.05 -16.00 16.50
## [21]  1.20  3.20 -16.90 20.30
```

26/39

Next, find the difference in cover values between 'Vaccviti' and 'Cladrang', but only in sites where the humus layer is less than 2. You will have to use the **varechem** dataset whose rownames ('sites') are the same as **varespec**.

27/39

Next, find the difference in cover values between 'Vaccviti' and 'Cladrang', but only in sites where the humus layer is less than 2. You will have to use the **varechem** dataset whose rownames ('sites') are the same as **varespec**.

```
data(varechem)
varespec[varechem$Humdepth <2, "Vaccviti"] - varespec[varechem$Humdepth <2, "Cladrang"]
```

```
## [1] -34.30 -56.25 -27.00 -25.07 -18.05 -16.00  3.20 -16.90
```

#Note we are using the data in one data frame to index another data frame.

28/39

Indexing Lists

Indexing lists

```
data(dune.env)
```

Let's make a new list.

```
dune.env$A1
```

```
## [1] 2.8 3.5 4.3 4.2 6.3 4.3 2.8 4.2 3.7 3.3 3.5 5.8 6.0 9.3 11.5  
## [16] 5.7 4.0 4.6 3.7 3.5
```

```
fivenum(dune.env$A1) # See ?fivenum
```

```
## [1] 2.80 3.50 4.20 5.75 11.50
```

Indexing lists

```
dune.env$Management
```

```
## [1] SF BF SF SF HF HF HF HF BF BF SF SF NM NM SF NM NM NM NM
## Levels: BF HF NM SF
```

```
table(dune.env$Management) # See ?table
```

```
##
## BF HF NM SF
## 3 5 6 6
```

31/39

Indexing lists

You can access the elements of a list in very similar ways. Let's make a list first.

```
dune.summ<-list(DATA=dune.env,
                SUMM.A1=fivenum(dune.env$A1),
                TAB.MANAGE=table(dune.env$Management))
str(dune.summ)
```

```
## List of 3
## $ DATA      : 'data.frame': 20 obs. of 5 variables:
## ..$ A1       : num [1:20] 2.8 3.5 4.3 4.2 6.3 4.3 2.8 4.2 3.7 3.3 ...
## ..$ Moisture : Ord.factor w/ 4 levels "1"<"2"<"4"<"5": 1 1 2 2 1 1 1 4 3 2 ...
## ..$ Management: Factor w/ 4 levels "BF","HF","NM",...: 4 1 4 4 2 2 2 2 1 ...
## ..$ Use       : Ord.factor w/ 3 levels "Hayfield"<"Haypastu"<...: 2 2 2 2 1 2 3 3 1 1 ...
## ..$ Manure    : Ord.factor w/ 5 levels "0"<"1"<"2"<"3"<...: 5 3 5 5 3 3 4 4 2 2 ...
## $ SUMM.A1     : num [1:5] 2.8 3.5 4.2 5.75 11.5
## $ TAB.MANAGE: 'table' int [1:4(1d)] 3 5 6 6
## ..- attr(*, "dimnames")=List of 1
## .. ..$ : chr [1:4] "BF" "HF" "NM" "SF"
```

32/39

Access list by name

```
names(dune.summ)

## [1] "DATA"      "SUMM.A1"    "TAB.MANAGE"

dune.summ$SUMM.A1 # result of `fivenum(dune.env$A1)`

## [1]  2.80  3.50  4.20  5.75 11.50

dune.summ$TAB.MANAGE # result of `table(dune.env$Management)`

##
## BF HF NM SF
##  3  5  6  6
```

33/39

Indexing Lists

We can also access a *copy* of elements within a list by item index.

```
dune.summ[2] # Access a copy of the second element in dune.summ

## $SUMM.A1
## [1]  2.80  3.50  4.20  5.75 11.50
```

- produces the item *and its name*

34/39

Indexing Lists

More useful is to access the *actual* elements. What if we want to access the median (the 3rd element) in SUMM.A1? Three main ways:

```
dune.summ$SUMM.A1[3]
```

```
## [1] 4.2
```

35/39

Indexing Lists: the infamous [[

Notice the big difference between these two:

```
dune.summ[2] # the element of the list
```

```
## $SUMM.A1
```

```
## [1] 2.80 3.50 4.20 5.75 11.50
```

```
dune.summ[[2]] # the actual vector
```

```
## [1] 2.80 3.50 4.20 5.75 11.50
```

Double brackets allow access to the vector itself, not just the element of the list.

36/39

Indexing Lists: the infamous [[

```
dune.summ[2][3]

## $<NA>
## NULL

dune.summ[[2]][3]

## [1] 4.2
```

37/39

Indexing lists

What if we want to access the **top 5 rows** in **DATA**? You try...

```
str(dune.summ)

## List of 3
## $ DATA      : 'data.frame': 20 obs. of 5 variables:
## ..$ A1       : num [1:20] 2.8 3.5 4.3 4.2 6.3 4.3 2.8 4.2 3.7 3.3 ...
## ..$ Moisture  : Ord.factor w/ 4 levels "1"<"2"<"4"<"5": 1 1 2 2 1 1 1 4 3 2 ...
## ..$ Management: Factor w/ 4 levels "BF","HF","NM",..: 4 1 4 4 2 2 2 2 1 ...
## ..$ Use       : Ord.factor w/ 3 levels "Hayfield"<"Haypastu"<..: 2 2 2 1 2 3 3 1 1 ...
## ..$ Manure    : Ord.factor w/ 5 levels "0"<"1"<"2"<"3"<..: 5 3 5 5 3 3 4 4 2 2 ...
## $ SUMM.A1     : num [1:5] 2.8 3.5 4.2 5.75 11.5
## $ TAB.MANAGE: 'table' int [1:4(1d)] 3 5 6 6
## .. attr(*, "dimnames")=List of 1
## .. ..$ : chr [1:4] "BF" "HF" "NM" "SF"
```

38/39

Indexing lists

What if we want to access the **top 5 rows** in **DATA**? First we need access to the actual data frame. After that, it's just like working with a normal data frame.

```
dune.summ[["DATA"]][1:5,]
```

##	A1	Moisture	Management	Use	Manure
## 1	2.8	1	SF Haypastu	4	
## 2	3.5	1	BF Haypastu	2	
## 3	4.3	2	SF Haypastu	4	
## 4	4.2	2	SF Haypastu	4	
## 5	6.3	1	HF Hayfield	2	